

1. Introduction

Le pipeline ETL est conçu pour traiter différents types de fichiers et les convertir en un format unique, puis de le sauvegarder.

Résumé des 3 cas de traitement			
Type fichier	Conversion	Extraction texte	Résultat
DOCX (Word)	docx4j → PDF natif	PDFBox direct	Texte structuré sans OCR
JPG / PNG	iText → PDF image	Doctr OCR (Python)	Texte reconnu par IA
PDF scanné	Copie directe (déjà PDF)	PDFBox → si vide → Doctr	Auto-détection natif ou scanné

2. Cas 1 — Fichier DOCX

Un fichier Word (.docx) est un fichier texte structuré, il est donc possible d'extraire le texte directement dans son format natif.

2.1 Flux de traitement

Étape	Action	Outil / Service	Résultat
1	Upload fichier .docx	Angular → FichierController	Fichier reçu sur le serveur
2	Conversion DOCX → PDF natif	PdfConversionService + docx4j	PDF avec texte natif lisible
3	Sauvegarde en base	FichierRepository	Entité Fichier créée
4	EXTRACT — lire le texte	EtIService + PDFBox 3.x (Loader.loadPDF)	Texte brut extrait directement
5	TRANSFORM — nettoyer	TextCleaningService	Texte nettoyé sans artefacts
6	LOAD — sauvegarder	OcrResultService.create()	OcrResult en base PostgreSQL

2.2 Caractéristiques

Le pipeline ETL est conçu pour traiter différents types de fichiers et les convertir en un format unique, puis de le sauvegarder. Log Spring Boot caractéristique : Stratégie CAS 1 : PDFBox (DOCX natif)

3. Cas 2 — Image JPG / PNG

Le pipeline ETL est conçu pour traiter différents types de fichiers et les convertir en un format unique, puis de le sauvegarder.

3.1 Flux de traitement

Ét ap e	Action	Outil / Service	Résultat
1	Upload fichier .jpg ou .png	Angular → FichierController	Image reçue sur le serveur
2	Conversion image → PDF	PdfConversionService + iText 5	PDF contenant l'image intégrée
3	Sauvegarde en base	FichierRepository	Entité Fichier créée
4	EXTRACT — appel Doctr OCR	EtlService → Doctrclientservice → POST /ocr	Service Python analyse l'image
5	Doctr traite le PDF	Python FastAPI + modèle Doctr	Texte + confiance retournés
6	TRANSFORM — nettoyer	TextCleaningService	Texte OCR nettoyé
7	LOAD — sauvegarder	OcrResultService.create()	OcrResult en base PostgreSQL

3.2 Service Python Doctr

Versionator Python sur <https://github.com/PyImageMagick/PyImageMagick> (traitement 80000 fichiers) se deux endpoints : GET /health et POST /ocr. Le service doit être démarré avant l'upload. Commande : uvicorn doctr_service:app -- Log Spring Boot caractéristique : Stratégie CAS 2 : Doctr OCR (Image) — type=jpg

4. Cas 3 — PDF scanné

Un fichier PDF peut être scanné ou généré à partir d'un document natif (Word, Excel, PowerPoint, etc.). La détection automatique permet de savoir si le PDF est natif ou scanné.

4.1 Détection automatique

Le service Doctr OCR permet de détecter si un PDF est natif ou scanné. Si le résultat est natif, le texte est extrait. Si le résultat est scanné, le texte est extrait et nettoyé.

Condition	Action	Exemple
Texte PDFBox >= 50 chars	PDF natif → garder le texte PDFBox	PDF généré depuis Word ou Excel
Texte PDFBox < 50 chars	PDF scanné → appeler Doctr OCR	Scan d'une fiche de paie ou CIN

4.2 Flux de traitement

Ét ap e	Action	Outil / Service	Résultat
1	Upload fichier .pdf	Angular → FichierController	PDF reçu sur le serveur
2	Copie directe (déjà PDF)	PdfConversionService (Files.copy)	PDF copié dans uploads/pdf/
3	Sauvegarde en base	FichierRepository	Entité Fichier créée

4	Essai PDFBox	EtlService.extraireAvecPdfBox()	Texte ou vide selon le type PDF
5a	Si texte >= 50 chars	Garder texte PDFBox	PDF natif — extraction directe
5b	Si texte < 50 chars	Appel Doctr OCR	PDF scanné — OCR nécessaire
6	TRANSFORM — nettoyer	TextCleaningService	Texte nettoyé
7	LOAD — sauvegarder	OcrResultService.create()	OcrResult en base PostgreSQL

Log Spring Boot caractéristique (natif) : PDF natif — X caractères extraits via PDFBox
Log Spring Boot caractéristique (scanné) : Texte PDFBox trop court (Y chars) → Doctr OCR'

5. Étape TRANSFORM — TextCleaningService

À cette étape, le texte est sauvegardé dans la table ocr_results via OcrResultService. Cette étape est

Transformation	Description
Caractères de contrôle	Supprime les caractères invisibles sauf \n et \t
Retours à la ligne	Normalise \r\n et \r en \n (Windows → Unix)
Lignes vides multiples	Réduit les sauts consécutifs à 2 maximum
Espaces multiples	Remplace les espaces/tabulations multiples par un seul espace
Trim par ligne	Supprime les espaces en début et fin de chaque ligne

6. Étape LOAD — OcrResult en base

Le texte nettoyé est sauvegardé dans la table ocr_results via OcrResultService. Cette étape est

Champ BDD	Valeur	Description
fichier_id	UUID du Fichier	Lien vers l'entité Fichier uploadée
texte_brut	Texte avant nettoyage	Sortie directe de PDFBox ou Doctr
texte_nettoye	Texte après nettoyage	Prêt pour GROQ et le chatbot RAG
statut	SUCCESS / FAILURE	Résultat du pipeline ETL
created_at	Timestamp automatique	Généré par @CreationTimestamp Hibernate

7. Comment vérifier le bon fonctionnement

Après chaque upload, trois vérifications permettent de confirmer que le pipeline ETL s'est bien

Vérification	Comment faire	Ce qu'on attend
1. Logs Spring Boot	Regarder la console IntelliJ	Voir les logs ETL dans l'ordre : EXTRACT → TRANSFORM → LOAD

2. Base de données	pgAdmin → table ocr_results	SELECT * FROM ocr_results ORDER BY created_at DESC LIMIT 5;
3. API REST	Postman : GET /api/ocr-results	Retourne la liste des résultats OCR avec texte_brut et texte_nettoye remplis